



Trial final exam with solutions - 2023

Introduction to python (Universiteit van Amsterdam)



messages.pdf_cover_qr_code_label

Question 1 – 1_Basics_Logic_2 – 357635.3.2

Suppose you already have: (i) a list of strings called **europe** which contain the names of all European countries, and (ii) a string variable called **destination**.

You want to write a program to advice Dutch residents on the documents they need to prepare before traveling:

- If they travel within Europe, your program should print "No passport needed. Bring your ID card."
- If they travel outside of Europe, the program should print "Please bring your passport."

Which of the following code segment will achieve what you need?

- A**
- ```
if destination in europe:
 print('No passport needed. Bring your ID card.')
else:
 print('Please bring your passport.')
```
- B**
- ```
if destination in europe:
    return 'No passport needed. Bring your ID card.'
elif destination not in europe:
    return 'Please bring your passport.'
```
- C**
- ```
for country in europe:
 if destination = country:
 print('No passport needed. Bring your ID card.')
 else:
 print('Please bring your passport.')
```
- D**
- ```
for country in europe:
    if destination != country:
        return 'Please bring your passport.'
    else:
        return 'No passport needed. Bring your ID card.'
```

Question 2 – 1_Basics_Scope_3 – 357612.1.5

Calling the function **func_1()** will produce an error. Which line of the following code segment produces the error?

```
def func_1(var):  
    def func_2(var):  
        var_1 = var * 2  
        return var_1  
    var_2 = var_1 - var  
    return var_2
```

- A** 'var_2 = var_1 - var'
This line produces the error because **func_2()** was not called and assigning the value to **var_1**, thus, **var_1** was referenced before being defined.
- B** 'var_1 = var * 2'
This line produces the error because Python is confused about which value **var** should have.
- C** 'def func_2(var):'
This line produces the error because the name **var** is already used for the argument of **func_1()** and cannot be used again as the name of the argument for **func_2()**.
- D** 'return var_2'
This line produces the error because one function cannot return two outputs.

Question 3 – 1_Basics_Variables_1 – 357573.2.0

What is the output of the following code segment?

```
def main(list_1):  
    list_2 = []  
    for item in list_1:  
        list_2.append(type(item))  
    return list_2  
  
main(['abc', 0, None, True, 123.456, 'False', {}])
```

- A** [str, int, NoneType, bool, float, str, dict]
- B** [str, int, NoneType, bool, float, bool, dict]
- C** [str, int, NoneType, bool, float, str, list]
- D** [str, float, NoneType, bool, int, str, dict]

Question 4 – 2-dictionaries – comprehension – 1 – 359259.1.10

You have to create a dictionary called **vowels_ASCII** where the keys are the lowercase vowels (a, e, i, o, u) and the values are their corresponding ASCII values.
The ASCII (American Standard Code for Information Interchange) is a character encoding standard that assigns unique numerical values to characters.

That is, if we execute the following command:

```
print(vowels_ASCII)
```

the output should be:

```
{ 'a': 97, 'e': 101, 'i': 105, 'o': 111, 'u': 117 }
```

Which of the following code segments work(s) as intended?

Hint: the `ord()` function returns a unique numerical value to a character. The function takes a unit-length text as an argument and returns the assigned unique numerical value.

For example:

```
ord('a')
```

returns:

```
97
```

- A** `vowels_ASCII = {vowel: ord(vowel) for vowel in 'aeiou'}`
- B** `vowels_ASCII = {vowel: ord(vowel) for vowel in 'aeiou' if vowel in 'aeiou'}`
- C** Both of the code segments work correctly.
- D** Neither of the code segments work correctly.

Question 5 – 2-dictionaries – manipulation – 1 – 358902.1.7

Suppose you have three variables called **player1_goals**, **player2_goals**, and **player3_goals**. Each of the variables contains a list of integers that represents the number of goals scored by the football players in five different matches:

```
player1_goals = [2, 1, 3, 2, 1]
player2_goals = [0, 1, 1, 3, 2]
player3_goals = [1, 0, 2, 1, 3]
```

You want to create a new dictionary called **match_goals** where the keys are the match numbers (starting from 1) and the values are a tuple containing the total goals scored by each player in that match.

So, if we execute the following command:

```
print(match_goals)
```

the output should be:

```
{1: (2, 0, 1), 2: (1, 1, 0), 3: (3, 1, 2), 4: (2, 3, 1), 5: (1, 2, 3)}
```

Which of the following code segments works as intended?

- A**

```
match_goals = {}
for i, (goals1, goals2, goals3) in enumerate(zip(player1_goals, player2_goals, player3_goals),
start=1):
    match_goals[i] = (goals1, goals2, goals3)
```
- B**

```
match_goals = {}
for i, (goals1, goals2, goals3) in enumerate(zip(player1_goals, player2_goals, player3_goals),
start=1):
    match_goals[i] = (goals1 + goals2 + goals3)
```
- C**

```
match_goals = {}
for i, (goals1, goals2, goals3) in enumerate(zip(player1_goals, player2_goals, player3_goals)):
    match_goals[i] = (goals1, goals2, goals3)
```
- D**

```
match_goals = {}
for i, (goals1, goals2, goals3) in enumerate(zip(player1_goals, player2_goals, player3_goals)):
    match_goals[i] = [goals1, goals2, goals3]
```

Question 6 – 2-dictionaries – slicing – 1 – 358888.1.1

You have the following dictionary that represents the monthly sales of a store:

```
monthly_sales = {'January': 1500, 'February': 2200, 'March': 1800, 'April': 2400, 'May': 2000, 'June': 2800}
```

You need to calculate the average sales for the first three months (January, February, and March). Which of the following code segments works as intended?

- A**

```
sales_subset = [monthly_sales[key] for key in list(monthly_sales.keys())[:3]]
average_sales = sum(sales_subset) / len(sales_subset)
print(average_sales)
```
- B**

```
sales_subset = monthly_sales[:3]
average_sales = sum(sales_subset) / len(sales_subset)
print(average_sales)
```
- C**

```
sales_subset = list(monthly_sales.items())[:3]
average_sales = sum(sales_subset) / len(sales_subset)
print(average_sales)
```
- D**

```
sales_subset = monthly_sales['January':'March']
average_sales = sum(sales_subset.values()) / len(sales_subset)
print(average_sales)
```

Question 7 – 2-list – comprehension – 2 – 358872.1.2

What is the output of the following code segment?

```
numbers = [1, 2, 3, 4, 5]
result = [x * 2 if x % 2 == 0 else x for x in numbers]
print(result)
```

- A** [1, 4, 3, 8, 5]
- B** [2, 4, 6, 8, 10]
- C** [1, 2, 3, 4, 5]
- D** [2, 2, 6, 4, 10]

Question 8 – 2-list – manipulation – 2 – 358859.1.3

What is the output of the following code segment?

```
list1 = [10, 20, 30, 40]
list2 = ['a', 'b', 'c', 'd']

result = []

for index, value in enumerate(list1):
    if index % 2 == 0:
        result.append(str(value) + list2[index])

print(result)
```

- A** ['10a', '30c']
- B** [10, 20, 30, 40]
- C** ['a', 'b', 'c', 'd']
- D** ['10a', '20b', '30c', '40d']

Question 9 – 2-list – slicing – 2 – 358866.1.3

Suppose you have a list called **mylist**. You want to extract every third element starting from index 1 (inclusive) to the last index (inclusive) from that list using the following line of code:

```
subset = mylist[__:__:__]
```

For example, if you have:

```
mylist = [10, 15, 20, 25, 30, 35, 40, 45]
```

then executing the following command:

```
print(subset)
```

should return:

```
[15, 30, 45]
```

What should the blanks be filled with?

- ☒ **A** 1:len(mylist):3
- ☐ **B** 0:len(mylist):3
- ☐ **C** 1:len(mylist)-1:3
- ☐ **D** 0:len(mylist)-1:3

Question 10 – 3_Function_Argument_1 – 357661.1.1

Suppose you have the following function:

```
def main(x = 0):  
    if x > 0:  
        return True  
    if x < 0:  
        return False
```

What will be printed by the following line of code?

```
print(main())
```

- ☐ **A** True
- ☐ **B** False
- ☐ **C** 0
- ☒ **D** None

Question 11 – 3_Function_Built-in_3 – 357817.1.2

Suppose you have a variable called **x** which contains a list of integers as follows:

```
x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Which of the following lines will produce an output that is different from the rest?

- A** `print(len(x))`
- B** `print(sorted(x, reverse=True)[0])`
- C** `print(x.index(9))`
- D** `print(sum(x[4:6]))`

Question 12 – 3_Function_Flexible-Argument_1 – 357693.1.7

You want to write a function called **main** which accepts a flexible number of integers as arguments.

The function should return a dictionary with the following key-value pairs:

- key **'sum'** has a value which is the sum of all the integers passed as arguments
- key **'pro'** has a value which is the multiplicative product of all the integers passed as arguments
- key **'pow'** has a value which is a list of all the squares of the integers passed as arguments

Which of the following lines of code will complete the function to give what you need?

```
def main(*args):  
    x = {}  
  
    # missing lines  
    # ...  
  
    return x
```

- A**

```
x['sum'] = sum(args)  
x['pro'] = 1  
x['pow'] = []  
for i in args:  
    x['pro'] *= i  
    x['pow'].append(i**2)
```
- B**

```
x['sum'] = sum(args)  
x['pro'] *= i for i in args  
x['pow'] = [i**2 for i in args]
```
- C**

```
for i in args:  
    x['sum'] = sum(args)  
    x['pro'] *= i  
    x['pow'] = [i**2 for i in args]
```
- D** None of the given options.

Question 13 – 6-string – fstring – 2 – 359330.1.2

You have the following objects loaded in Python:

```
item = "laptop"  
price = 1299.99
```

You need to print the following sentence:

The laptop is priced at \$1299.99.

Which of the following lines of code works as intended?

- A**

```
message = "The {0} is priced at ${1}.".format(item, price)  
print(message)
```
- B**

```
message = "The {0} is priced at ${1}.".format(item, price)  
print(message)
```
- C**

```
message = "The {item} costs ${price}.".format(price, item)  
print(message)
```
- D**

```
message = "The {item} costs ${price}.".format(price, item)  
print(message)
```

Question 14 – 7-pandas – easy – 1 – 359417.2.2

Suppose you have run the following block of code:

```
import pandas as pd  
data = {'A': [1, 2, 3],  
        'B': [4, 5, 6],  
        'C': [7, 8, 9]}  
df = pd.DataFrame(data)
```

Which of the following lines of code give(s) the following output?

```
0    4  
1    5  
2    6  
Name: B, dtype: int64
```

- A**

```
df.loc[:, 'B']
```
- B**

```
df['B'].loc[:]
```
- C**

```
df['B']
```
- D** All of the given lines.

Question 15 – 8_Datetime_Easy_1 – 358039.1.10

Which code segment gives the same output as the following?

```
date = datetime.strptime("05.12.2023", "%m.%d.%Y")
print((date + timedelta(days=-10)).strftime("%d-%m-%Y"))
```

You may assume that the datetime and timedelta classes are already imported from the datetime module.

- A** `print(datetime.strptime('22-05-2023', '%d.%m.%Y'))`
- B** `print('02-05-2023')`
- C** `print(datetime.strptime('05.22.2023', '%d-%m-%Y'))`
- D** `date = datetime.strptime('10/05/2023', '%d/%m/%Y')`
 `print((date + timedelta(days=5)).strftime('%m-%d-%Y'))`

Question 16 – 9_OOP_Easy_3 – 358035.3.1

You have the following definition for a class called **Shape**:

```
class Shape:
    def __init__(self, height, length):
        self.height = height
        self.length = length
        self.area   = self.height * self.length
```

What would be the output if we execute the following commands?

```
print(Shape(5, 3).area == Shape(1, 15).length)
```

- A** `True`
- B** `False`
- C** `An error`
- D** `None of the given options.`

Question 17 – 6-string – operations – 2 – 359328.2.0

Write a function called **shuffle_sentence** that takes a string as an argument and returns a new string where the words are shuffled **randomly**.

For example, if your function is called as:

```
shuffle_sentence("Hello, how are you today?")
```

then it could return a shuffled string like:

```
'how today? you Hello, are'
```

Which of the following programs work(s) according to the requirements?

Hints:

The **shuffle()** function is a method from the random module. The **shuffle()** method takes a sequence, like a list, and reorganizes the order of the items. This method changes the original list; it does not return a new list.

The **reversed()** function computes the reverse of a given sequence object and returns it in the form of a list.

A

```
def shuffle_sentence(sentence):  
    words = sentence.split()  
    return ' '.join(reversed(words))
```

B

```
import random  
  
def shuffle_sentence(sentence):  
    words = sentence.split()  
    random.shuffle(words)  
    return ' '.join(words)
```

C Both of the programs work.

D Neither of the programs work.

Question 18 – 6-strings – find & replace – 1 – 359277.2.4

Consider the following line of code:

```
text = "The demand for oil has been increasing. Natural gas is a crucial commodity in many industries. The price of oil has also been rising."
```

You want to create a new string called **new_text** where all occurrences of "oil" are replaced by "natural gas".

So:

```
print(new_text)
```

should return:

```
"The demand for natural gas has been increasing. Natural gas is a crucial commodity in many industries. The price of natural gas has also been rising."
```

Which of the following code segments achieve(s) what you want?

A

```
new_text = ' '.join([word if word != 'oil' else 'natural gas' for word in text.split()])
```

B

```
new_text = text.replace("oil", "natural gas")
```

C

```
new_text = text.split()  
for index, word in enumerate(new_text):  
    if word == 'oil':  
        new_text[index] = 'natural gas'  
new_text = ' '.join(new_text)
```

D All of the given options work as intended.

Question 19 – 7-pandas – hard – 1 – 359443.1.4

Consider the following pandas DataFrame called **df**:

	Name	Age	Gender	Salary
0	John	25	Male	50000
1	Jane	30	Female	65000
2	Alex	35	Male	80000
3	Lisa	40	Female	70000
4	Mark	28	Male	55000

You need to perform a series of operations on this DataFrame.

- Operation 1: Select all rows where the age is greater than 30 and the gender is "Male".
- Operation 2: Create a new column called **Sex_abbr** of which the values should be "F" if gender is "Female" and "M" if gender is "Male".
- Operation 3: Calculate the average salary of all employees.

Which of the following blocks of code performs these operations as intended?

- A**
- ```
df_operation1 = df[(df['Age'] > 30) & (df['Gender'] == 'Male')]
df['Sex_abbr'] = ['M' if x == 'Male' else 'F' for x in df['Gender']]
df_operation3 = df['Salary'].mean()
```
- B**
- ```
df_operation1 = df.loc(df['Age'] > 30) & df.loc(df['Gender'] == 'Male')
df['Sex_abbr'] = ['M' if x == 'Male' else 'F' for x in df['Gender']]
df_operation3 = df['Salary'].mean()
```
- C**
- ```
df_operation1 = df.loc(df['Age'] > 30) & df.loc(df['Gender'] == 'Male')
df['Sex_abbr'] = df['Gender'].map(lambda x: 'M' if x == 'Male' else 'F')
df_operation3 = df['Salary'].sum()/len(df)
```
- D**
- ```
df_operation1 = df[df['Age'] > 30 & df['Gender'] == 'Male']
df['Sex_abbr'].map(lambda x: 'M' if x == 'Male' else 'F' for x in df['Gender'])
df_operation3 = df['Salary'].sum()/len(df)
```

Question 20 – 7-pandas – lambda – 1 – 359440.1.7

Consider the following pandas DataFrame called **df**:

	Name	Performance_review	Salary
0	John	2	50000
1	Jane	4	65000
2	Alex	3	80000
3	Lisa	2	70000
4	Mark	1	55000

You want to create a new column called **Bonus** that compute each employee's bonus based on their salary and performance review using the following formula:

- if the performance review is less than 4, the bonus is 0.
- if the performance review is 4 or 5, the bonus is 5% of the salary.

Which of the following code snippets works as intended?

- A**
- ```
df['Bonus'] = df.apply(lambda row: row['Salary']*0.05 if row['Performance_review']>=4 else 0,
axis=1)
```
- B**
- ```
df['Bonus'] = df['Salary'].apply(lambda x: x*0.05 if df['Performance_review']>=4 else 0)
```
- C**
- ```
df['Bonus'] = map(df['Salary'].apply(lambda x: x*0.05 if df['Performance_review']>4 else 0))
```
- D**
- ```
df['Bonus'] = df['Salary'].map(lambda row: row*0.05 if df['Performance_review']>=4 else 0)
```

Question 21 – 8_Datetime_Function_3 – 358095.4.5

You want to create a function called **main** which take an integer and a flexible number of strings as arguments. The strings represent dates in the format of DD MM YYYY. The day, month and year can be separated by different characters.

Your function should look for the string inputs which represent dates as DD-MM-YYYY or DD/MM/YYYY and return those strings in the format of DD.MM.YYYY in a list. Furthermore, the dates represented by the strings should be shifted by the amount of days as given by the integer argument.

For example, if we call your function as:

```
main(3, '25/06/2003', '26.06.2003', '27-06-2003', '28!06!2023', '29-06-2023', '30.06.2023')
```

the output should be:

```
['28.06.2003', '30.06.2003', '02.07.2023']
```

Take a look at the following code segment. Does it do what is intended? If not, then why not?

```
def main(delta, *args):
    dates = []
    for string in args:
        if '-' in string:
            dates.append(datetime.strptime(string, '%d-%m-%Y') + timedelta(days=delta))
        elif '/' in string:
            dates.append(datetime.strptime(string, '%d/%m/%Y') + timedelta(days=delta))
        else:
            dates.append(None)
    return [date for date in dates if type(date) != None]
```

- A** The code segment works as intended.
- B** This code segment does not work because the function will return a list of datetime objects, not a list of strings representing dates.
- C** This code segment does not work because the **strptime()** method cannot be applied to **NoneType** object.
- D** This code segment produces an error because you cannot concatenate strings and timedelta.

Question 22 – 8_Datetime_Hard_2 – 358078.3.9

You want to define a class called **Calendar**, which has four attributes that are initialized at the time of object construction:

1. The **'event'** attribute can be any string,
2. The **'date'** attribute is a string representing a date as 'DD-MM-YYYY',
3. The **'start'** attribute is a string representing a time as 'HH:MM', and
4. The **'length'** attribute is an integer representing the amount of minutes.

The class should also have a method called **get_details()** that gives a description of Calendar objects.

For example, if we execute the following commands;

```
event1 = Calendar('Coffee meeting', '01-06-2023', '9:30', 20)
print(event1.get_details())
```

the output should be:

Coffee meeting: 01-06-2023, 9:30-9:50.

Which of the following code segments gives you what you want?

- A**
- ```
class Calendar():
 def __init__(self, event, date, start, length):
 self.event = event
 self.date = datetime.strptime(date, '%d-%m-%Y')
 self.start = datetime(self.date.year, self.date.month, self.date.day, int(start.split(":")
[0]), int(start.split(":")[1]))
 self.end = self.start + timedelta(minutes=length)
 def get_details(self):
 return f'{self.event}: {self.date.strftime("%d-%m-%Y")}, {self.start.hour}:
{self.start.minute}-{self.end.hour}:{self.end.minute}.'
```
- B**
- ```
class Calendar():
    def __init__(self, event, date, start, length):
        self.event = event
        self.date = date
        self.start = datetime(self.date, hour=start.split(':')[0], minute=start.split(':')[1])
        self.length = timedelta(minutes=length)
        self.end = self.start + self.length
    def get_details(self):
        return f'{self.event}: {self.date}, {self.start.hour}:{self.start.minute}-{self.end.hour}:
{self.end.minute}.'
```
- C**
- ```
class Calendar():
 def __init__(self, event, date, start, length):
 self.event = event
 self.date = date
 self.start = datetime(self.date, hour=start.hour, minute=start.minute)
 self.end = self.start + timedelta(minutes=length)
 def get_details(self):
 return f'{self.event}: {self.date}, {self.start.hour}:{self.start.minute}-{self.end.hour}:
{self.end.minute}.'
```
- D**
- ```
class Calendar():
    def __init__(self, event, date, start, length):
        self.event = event
        self.date = datetime.strptime(date, '%d-%m-%Y')
        self.start.hour = start.hour
        self.start.minute = start.minute
        self.end = self.start + timedelta(minutes=length)
    def get_details(self):
        return f'{self.event}: {self.date.strftime("%d-%m-%Y")}, {self.start.hour}:
{self.start.minute}-{self.end.hour}:{self.end.minute}.'
```

Question 23 – 9_OOP_Function_2 – 357882.3.3

You have the following definition for a class called **Employee**:

```
class Employee:
    def __init__(self, name, role, salary):
        self.name = name
        self.role = role
        self.salary = salary
    # ...
```

You want to add three methods to this class:

1. The **holiday_bonus()** method which computes and returns the holiday bonus, which is 8.0% of the yearly salary,
2. The **year_end_bonus()** method which computes and returns the year end bonus, which is 8.3% of the yearly salary
3. The **payment_overview()** method which returns a payment overview for the employee using the following format:
[Name] receives \${salary} monthly, \${holiday bonus} in May and \${year end bonus} in December.

For example, if we execute the following commands:

```
employee1 = Employee('John Smith', 'HR preresentative', 2500)
print(employee1.holiday_bonus())
print(employee1.year_end_bonus())
print(employee1.payment_overview())
```

the output should be:

```
2400.0
2490.0
John Smith receives $2500 monthly, $2400.0 in May and $2490.0 in December.
```

Which of the following code segments should you place on the blank in the class definition to achieve what you want?

- A**
- ```
def holiday_bonus(self):
 return self.salary * 12 * 0.080
def year_end_bonus(self):
 return self.salary * 12 * 0.083
def payment_overview(self):
 return f'{self.name} receives ${self.salary} monthly, ${self.holiday_bonus()} in May and ${self.year_end_bonus()} in December.'
```
- B**
- ```
def payment_overview(self):
    return f'{self.name} receives ${self.salary} monthly, ${self.salary*12*0.08} in May and ${self.salary*12*0.083} in December.'
```
- C**
- ```
def holiday_bonus():
 self.holiday_bonus = self.salary * 12 * 0.080
def year_end_bonus():
 self.year_end_bonus = self.salary * 12 * 0.083
def payment_overview():
 return f'{self.name} receives ${self.salary} monthly, ${self.holiday_bonus} in May and ${self.year_end_bonus} in December.'
```
- D**
- ```
def payment_overview(self):
    self.holiday_bonus = self.salary * 12 * 0.080
    self.year_end_bonus = self.salary * 12 * 0.083
    return f'{self.name}' receives ${self.salary} monthly, ${self.holiday_bonus} in May and ${self.year_end_bonus} in December.'
```

Question 24 – 9_OOP_Hard_2 – 358026.3.10

You have the following definitions for a class called **Car**:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def get_description(self):
        return f"{self.brand}, {self.model}"
```

You want to create another class called **Garage**, which has two attributes that are initialized during object construction:

- The '**capacity**' attribute, which is an integer and must be passed to the object constructor.
- The '**cars**' attribute, which is an empty list.

The **Garage** class should also have two methods:

1. The **add_car()** method, which adds Car objects to a Garage object. New Car objects can only be added if there is still capacity in the Garage object. If not, the method should return "Capacity reached."
2. The **check_garage()** method, which reports the Car objects that have been added to the Garage object. The method should return a dictionary in which the keys are the counts of Car objects starting from 1 and the values are the Car objects' descriptions.

For example, if we execute the following commands:

```
my_garage = Garage(2)
my_garage.add_car(Car("Toyota", "Camry"))
my_garage.add_car(Car("Honda", "Accord"))
print(my_garage.check_garage())
```

the output should be:

```
{1: 'Toyota, Camry', 2: 'Honda, Accord'}
```

Which of the following code segments achieve(s) what you want?

A

```
class Garage:
    def __init__(self, capacity, cars = []):
        self.capacity = capacity

    def add_car(self, car):
        self.cars += car if len(self.cars) < self.capacity else return 'Capacity reached.'

    def check_garage(self, car):
        return {index: car.get_description() for index, car in enumerate(self.cars, start=1)}
```

B

```
class Garage:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cars = []

    def add_car(self, car):
        if len(self.cars) < self.capacity:
            self.cars.append(car)
        else:
            return 'Capacity reached.'

    def check_garage(self):
        output = {}
        for index, car in enumerate(self.cars, start=1):
            output[index] = car.get_description()
        return output
```

C Both of the above will work.

D None of the above will work.

